In [ ]:
```
import marimo as mo
import random
import time
import itertools
from collections import deque
from z3 import (Solver, Bool, BoolVal, And, Or, Not, Xor, Implies, PbEq,
                AtMost, AtLeast, sat, unsat, unknown, is_true, is_false)
```

TP2 — Exercício 1: *Game of Life* finito como máquina de estados finita

Lógica Computacional 2026/27

Enunciado

O espaço de células usa índices $(i, j) \in \{0, \ldots, N+1\}^2$, com $N > 16$:

tipo	células	quantidade	comportamento
normais	$1 \leq i, j \leq N$	N^2	evoluem pela regra **B3/S23**
iniciais	$i = 0$ ou $j = 0$	$2N + 3$	constantes; sorteadas no estado inicial (viva com probabilidade p)
finais	não iniciais, com $i = N + 1$ ou $j = N + 1$	$2N + 1$	constantemente mortas

- **Vizinhança:** (i_0, j_0) e (i_1, j_1) são vizinhas sse $\max(|i_0 - i_1|, |j_0 - j_1|) = 1$. Toda a célula normal tem exatamente 8 vizinhos.
- **Estado inicial:** as células normais com $i, j \leq 3$ estão vivas e as restantes mortas.
- **Estado de erro:** todas as células normais mortas.
- **Transições:** o parâmetro `MODO` escolhe uma de duas relações.
  - `"async"`: em cada transição muda **uma e só uma** célula normal.
  - `"sync"`: em cada transição mudam **todas** as células normais em simultâneo.

> **Nota sobre a contagem.** As células $(0, N+1)$ e $(N+1, 0)$ têm um índice 0 e outro $N+1$. A expressão "*finais que não são iniciais*" indica que são contadas como **iniciais**. Daí $2N+3$ iniciais e $2N+1$ finais. Para mudar esta convenção basta alterar as listas `iniciais` / `finais` em `GameOfLife.__init__`.

Parâmetros

Todos os parâmetros do problema estão nesta célula. `MODO` seleciona a relação de transição.

A regra B/S também é um parâmetro (`NASCE` , `SOBREVIVE`). Assim é fácil pereceber o impacto de pequenas modificações ao enunciado.

A profundidade das verificações **não é adivinhada**: no modo `sync` é calculada a partir do número de estados alcançáveis (ver a célula *Profundidade*). No modo `async` usa-se `K_ASYNC` , que só é preciso para P1 e para a leitura LTL da P2.

In [ ]:
```
N          = 17                     # dimensão da grelha de células normais (N > 16)
MODO       = "sync"                 # "sync": todas as células mudam; "async": uma e só uma muda
P_VIVA     = 0.5                    # probabilidade de uma célula inicial estar viva
SEED       = 2026                   # semente do sorteio das células iniciais
NASCE      = {3}                    # regra B3 ...
SOBREVIVE  = {2, 3}                 # ... /S23
VIVAS_T0   = lambda i, j: i <= 3 and j <= 3   # células normais vivas no estado inicial
K_ASYNC    = 12                     # profundidade no modo async (P1 e P2 LTL)
TIMEOUT_MS = 120_000                # tempo máximo de cada chamada ao solver (ms)
TIMEOUT_PASSO_MS = 2_000            # tempo máximo do passo indutivo (atalho opcional)
```

1. A máquina de estados finita

Variáveis de estado. Um booleano $s_{i,j}$ por célula normal.

Células constantes. As células iniciais e finais **não** são variáveis de estado. São constantes partilhadas por todos os estados:

- uma célula final é `False` ;
- uma célula inicial pode ser concreta (`BoolVal` , sorteada) ou **simbólica** (uma constante `Bool` livre e rígida no tempo). No caso simbólico, uma só verificação cobre as 2^{2N+3} configurações possíveis.

Regra (codificação booleana). Seja $\#_c(s) = k$ a restrição pseudo-booleana "exatamente k vizinhos de c estão vivos" (`PbEq`). Então

$$\rho_c(s) \;\equiv\; \bigvee_{k \in \text{NASCE}} \#_c(s) = k \;\vee\; \Big(s_c \wedge \bigvee_{k \in \text{SOBREVIVE}} \#_c(s) = k\Big).$$

Os vizinhos que são constantes (moldura sorteada) não entram no `PbEq` : são somados à parte e só ajustam o valor de k. Assim a regra é uma fórmula **puramente booleana**, sem inteiros (`Sum(If(...))`). No caso concreto `sync` o sistema é determinista e o solver resolve quase só por propagação.

Estado inicial. $I(s) \equiv \bigwedge_c s_c = [c_i \le 3 \wedge c_j \le 3]$.

Relação de transição. Seja $m_c \equiv s_c' \oplus s_c$ ("a célula c muda").

`MODO`	$T(s, s')$
`sync`	$\bigwedge_c s_c' = \rho_c(s)$
`async`	$\bigwedge_c (m_c \rightarrow s_c' = \rho_c(s)) \ \wedge \ \mathrm{AtMost}(m, 1) \ \wedge \ \left(\mathrm{AtLeast}(m, 1) \vee \bigwedge_c \rho_c(s) = s_c\right)$

No modo `async` muda exatamente uma célula: uma das que **podem** mudar segundo a regra, escolhida de forma não determinista. Quando nenhuma célula pode mudar, o estado é estável e repete-se (*stuttering*). Sem isso, a relação teria estados de bloqueio e não seria total.

A codificação `async` é **linear** no número de células: cada célula aparece um número fixo de vezes. A versão "para cada c, todas as outras ficam iguais" tem tamanho quadrático (N^4 termos), o que para $N = 17$ dá mais de 80 000 igualdades por transição.

Estado de erro. $E(s) \equiv \bigwedge_c \neg s_c$.

In [ ]:
```
class GameOfLife:
    def __init__(self, N, modo, iniciais=None, nasce=NASCE, sobrevive=SOBREVIVE,
vivas0=VIVAS_T0):
        assert N > 3 and modo in ("sync", "async")
        self.N, self.modo, self.vivas0 = N, modo, vivas0
        self.nasce, self.sobrevive = set(nasce), set(sobrevive)
        R = range(N + 2)
        self.normais  = [(i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
        self.iniciais = [(i, j) for i in R for j in R if i == 0 or j == 0]
        self.finais   = [(i, j) for i in R for j in R
                         if (i, j) not in self.iniciais and (i == N + 1 or j == N
+ 1)]
        if iniciais is None:               # simbólicas: todas as configurações de
uma vez
            self.cte = {c: Bool(f"ini_{c[0]}_{c[1]}") for c in self.iniciais}
        else:                              # concretas (sorteadas)
            self.cte = {c: BoolVal(bool(iniciais[c])) for c in self.iniciais}
        self.cte.update({c: BoolVal(False) for c in self.finais})
        self.viz = {c: [(c[0] + a, c[1] + b) for a in (-1, 0, 1) for b in (-1, 0,
1) if (a, b) != (0, 0)]
                    for c in self.normais}

    # ---- variáveis de estado: uma cópia por instante ----
    def declare(self, k):
        return {c: Bool(f"c{c[0]}_{c[1]}_t{k}") for c in self.normais}

    def val(self, s, c):
        return s[c] if c in s else self.cte[c]

    # ---- "exatamente k vizinhos vivos" como restrição pseudo-booleana ----
    def vizinhos_vivos(self, s, c, k):
```

```
        literais, fixos = [], 0
        for d in self.viz[c]:
            x = self.val(s, d)
            if is_true(x):
                fixos += 1                  # vizinho constante vivo: conta à parte
            elif not is_false(x):
                literais.append(x)          # variável de estado ou inicial
simbólica
        r = k - fixos
        if r < 0 or r > len(literais):
            return BoolVal(False)
        if not literais:
            return BoolVal(True)            # r == 0 e não há mais vizinhos
        return PbEq([(x, 1) for x in literais], r)

    def regra(self, s, c):
        nasce = Or([BoolVal(False)] + [self.vizinhos_vivos(s, c, k) for k in
self.nasce])
        sobrevive = Or([BoolVal(False)] + [self.vizinhos_vivos(s, c, k) for k in
self.sobrevive])
        return Or(nasce, And(s[c], sobrevive))

    def init(self, s):
        return And([s[c] == BoolVal(bool(self.vivas0(*c))) for c in self.normais])

    def trans(self, s, p):
        rho = {c: self.regra(s, c) for c in self.normais}
        if self.modo == "sync":
            return And([p[c] == rho[c] for c in self.normais])
        muda = [Xor(p[c], s[c]) for c in self.normais]
        return And(
            And([Implies(m, p[c] == rho[c]) for m, c in zip(muda, self.normais)]),
# quem muda segue a regra
            AtMost(*muda, 1),
# no máximo uma muda
            Or(AtLeast(*muda, 1),
# pelo menos uma muda...
               And([rho[c] == s[c] for c in self.normais])))
# ...ou o estado é estável

    def erro(self, s):
        return And([Not(s[c]) for c in self.normais])

    # ---- visualização: '#'/'.' inicial viva/morta, 'X'/'-' normal viva/morta, '
' final ----
    def grelha(self, m, s):
        linhas = []
        for i in range(self.N + 2):
            r = ""
            for j in range(self.N + 2):
                c = (i, j)
                if c in s:
                    r += "X" if is_true(m.eval(s[c], model_completion=True)) else
"-"
                elif c in self.finais:
                    r += " "
```

```
                else:
                    r += "#" if is_true(m.eval(self.cte[c],
model_completion=True)) else "."
            linhas.append(r)
        return linhas


def mostra_traco(M, m, tr, cols=16):
    gs = [M.grelha(m, s) for s in tr]
    w = M.N + 2
    for b in range(0, len(gs), cols):
        bloco = gs[b:b + cols]
        print("   ".join(f"t={b + i}".ljust(w) for i in range(len(bloco))))
        for r in range(w):
            print("   ".join(g[r] for g in bloco))
        print()


def sorteia_iniciais(N, p, seed):
    rng = random.Random(seed)
    R = range(N + 2)
    return {c: rng.random() < p for c in [(i, j) for i in R for j in R if i == 0
or j == 0]}
```

In [ ]:

```
def explicito(N, iniciais, modo, nasce=NASCE, sobrevive=SOBREVIVE,
vivas0=VIVAS_T0):
    """Enumeração explícita (sem Z3) dos estados alcançáveis, por BFS.
    Os estados são bitmasks das N^2 células normais."""
    normais = [(i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
    idx = {c: k for k, c in enumerate(normais)}
    vizn, cte = [], []
    for (i, j) in normais:
        vs = [(i + a, j + b) for a in (-1, 0, 1) for b in (-1, 0, 1) if (a, b) !=
(0, 0)]
        vizn.append([idx[v] for v in vs if v in idx])
        cte.append(sum(1 for v in vs if iniciais.get(v, False)))      # finais
contam 0
    s0 = sum(1 << idx[c] for c in normais if vivas0(*c))

    def regra(s, k):
        n = cte[k] + sum((s >> d) & 1 for d in vizn[k])
        return 1 if (n in nasce or (((s >> k) & 1) and n in sobrevive)) else 0

    def succ(s):
        if modo == "sync":
            return [sum(regra(s, k) << k for k in range(N * N))]
        out = [s ^ (1 << k) for k in range(N * N) if regra(s, k) != (s >> k) & 1]
        return out or [s]

    dist = {s0: 0}; q = deque([s0])
    while q:
        s = q.popleft()
        for t in succ(s):
            if t not in dist:
                dist[t] = dist[s] + 1; q.append(t)
    sempre_viva = [normais[k] for k in range(N * N) if all((s >> k) & 1 for s in
```

```
dist)]
    sempre_morta = [normais[k] for k in range(N * N) if all(not (s >> k) & 1 for s
in dist)]
    return dict(alcancaveis=len(dist), profundidade=max(dist.values()),
erro=dist.get(0),
                sempre_viva=sempre_viva, sempre_morta=sempre_morta)
```

In [ ]:
```
INICIAIS = sorteia_iniciais(N, P_VIVA, SEED)
M_conc = GameOfLife(N, MODO, INICIAIS)     # células iniciais sorteadas
M_simb = GameOfLife(N, MODO)               # células iniciais simbólicas (todas as
configurações)

print(f"N={N}  modo={MODO}  normais={len(M_conc.normais)}  iniciais=
{len(M_conc.iniciais)}  finais={len(M_conc.finais)}")
print("Estado inicial com as células iniciais sorteadas:\n")
_s0 = M_conc.declare(0); _sol = Solver(); _sol.add(M_conc.init(_s0)); _sol.check()
mostra_traco(M_conc, _sol.model(), [_s0])
```

```
N=17  modo=sync  normais=289  iniciais=37  finais=35
Estado inicial com as células iniciais sorteadas:

t=0
#...##........#.#.#
.XXX--------------
.XXX--------------
.XXX--------------
#-----------------
#-----------------
.-----------------
.-----------------
.-----------------
.-----------------
.-----------------
#-----------------
#-----------------
.-----------------
#-----------------
#-----------------
.-----------------
.-----------------
.
```

Profundidade das verificações (`K_MAX`)

No modo `sync` com a moldura sorteada o sistema é **determinista**: há uma única trajetória, que percorre A estados distintos (os alcançáveis) e depois repete. A enumeração explícita calcula A e a profundidade $\max(\text{dist}) = A - 1$ sem usar o Z3, e é barata mesmo para $N > 16$, porque o modo `sync` tem poucos estados alcançáveis.

Não existe nenhum caminho simples com $A + 1$ estados. Por isso $K_{MAX} = A + 1$ chega para todas as provas por **diâmetro de recorrência**, e o BMC até K_{MAX} já visita todos os estados alcançáveis.

No modo `async` o número de estados alcançáveis é enorme e não há um limite útil deste tipo. Usa-se `K_ASYNC`, que basta para encontrar os contra-exemplos de P1 e da leitura LTL da P2.

In [ ]:
```
_t0 = time.time()
TRAJ_SYNC = explicito(N, INICIAIS, "sync")
K_SYNC = TRAJ_SYNC["alcancaveis"] + 1
K_MAX = K_SYNC if MODO == "sync" else K_ASYNC
print(f"sync: {TRAJ_SYNC['alcancaveis']} estados alcançáveis, profundidade
{TRAJ_SYNC['profundidade']} "
      f"({time.time() - _t0:.1f}s)  ->  K_SYNC = {K_SYNC}")
print(f"modo {MODO}: K_MAX = {K_MAX}")
```

```
sync: 9 estados alcançáveis, profundidade 8 (0.0s)  ->  K_SYNC = 10
modo sync: K_MAX = 10
```

2. Métodos de verificação

- **BMC** (*bounded model checking*): procura um traço $s_0 \dots s_k$ com $I(s_0) \wedge \bigwedge T(s_i, s_{i+1}) \wedge \neg P(s_k)$. Se o encontra, é um **contra-exemplo**.
- **k-indução com caminhos simples.** O passo indutivo exige estados $s_0, \dots, s_k$ distintos dois a dois. Acrescenta-se o teste do **diâmetro de recorrência**: se não existe nenhum caminho simples com k estados a partir de I, todos os estados alcançáveis já foram visitados pelo caso base. Com estas duas condições, a k-indução é **completa** para sistemas finitos: termina sempre com prova ou contra-exemplo.
- **Laços (*lassos*).** Uma propriedade de animação como $\mathrm{GF}\, c$ é refutada por um traço finito que fecha um ciclo, $s_0 \dots s_l \dots s_k$ com $s_k = s_l$. Esse traço representa uma execução infinita.

Timeouts. Todos os `Solver()` têm um limite de tempo (`TIMEOUT_MS`). Se o Z3 responde `unknown`, o resultado é **inconclusivo**. Nunca é tratado como "invalido" nem como prova.

A única exceção é o **passo indutivo** da k-indução. Como não tem `init`, os estados são livres e a pesquisa pode ser cara. Mas é só um atalho, porque a prova completa vem do caso base e do diâmetro. Por isso tem um limite curto (`TIMEOUT_PASSO_MS`): se responder `unknown`, não se conclui nada e passa-se ao k seguinte.

In [ ]:
```
def novo_solver(timeout=TIMEOUT_MS):
    s = Solver()
    s.set("timeout", timeout)
    return s


def unroll(M, k):
    tr = [M.declare(i) for i in range(k + 1)]
    return tr, [M.trans(tr[i], tr[i + 1]) for i in range(k)]


def distintos(M, a, b):
    return Or([a[c] != b[c] for c in M.normais])
```

```
def bmc(M, bad, K):
    """Procura o contra-exemplo mais curto até K. Devolve um dicionário com
    resultado = "contra-exemplo" (com k, modelo e traço),
    (até K) "inconclusivo" (o solver respondeu unknown no passo k)."""
    s = novo_solver(); tr = [M.declare(0)]; s.add(M.init(tr[0]))
    for k in range(K + 1):
        if k > 0:
            tr.append(M.declare(k)); s.add(M.trans(tr[k - 1], tr[k]))
        s.push(); s.add(bad(tr[k]))
        r = s.check()
        if r == sat:
            return dict(resultado="contra-exemplo", k=k, modelo=s.model(),
traco=tr)
        s.pop()
        if r == unknown:
            return dict(resultado="inconclusivo", k=k)
    return dict(resultado="inconclusivo", k=K)


def kinducao(M, P, K):
    # k-indução com caminhos simples + diâmetro de recorrência (completa para
FSM).
    for k in range(1, K + 1):
        tr, T = unroll(M, k - 1)
        s = novo_solver(); s.add(M.init(tr[0]), *T, Or([Not(P(x)) for x in tr]))
        r = s.check()
        if r == sat:
            return "FALSO", k
        if r == unknown:
            return "INCONCLUSIVO (timeout)", k
        s = novo_solver(); s.add(M.init(tr[0]), *T,
                                 *[distintos(M, tr[i], tr[j]) for i in range(k)
for j in range(i + 1, k)])
        r = s.check()
        if r == unsat:
            return "PROVADO (diâmetro de recorrência)", k
        if r == unknown:
            return "INCONCLUSIVO (timeout)", k
        # Passo indutivo: sem init, os estados são livres e a pesquisa pode ser
cara.
        # É só um atalho (a prova completa vem do diâmetro), por isso tem um
timeout
        # curto: se responder unknown não se conclui nada e passa-se ao k
seguinte.
        tr, T = unroll(M, k)
        s = novo_solver(TIMEOUT_PASSO_MS)
        s.add(*T, *[P(x) for x in tr[:-1]], Not(P(tr[-1])),
              *[distintos(M, tr[i], tr[j]) for i in range(k + 1) for j in range(i
+ 1, k + 1)])
        if s.check() == unsat:
            return "PROVADO (passo indutivo)", k
    return "INCONCLUSIVO", K
```

```
def laco(M, K, celulas=None):
    """Laço s0..sl..sk (sk = sl) em que alguma célula normal é constante no ciclo.
    Devolve um dicionário com resultado = "contra-exemplo", "inconclusivo"
    (até K) ou "inconclusivo"."""
    celulas = celulas or M.normais
    for k in range(1, K + 1):
        tr, T = unroll(M, k)
        s = novo_solver(); s.add(M.init(tr[0]), *T)
        ops = []
        for l in range(k):
            fecha = And([tr[k][c] == tr[l][c] for c in M.normais])
            cte = Or([And([tr[i][c] == tr[l][c] for i in range(l, k)]) for c in
celulas])
            ops.append(And(fecha, cte))
        s.add(Or(ops))
        r = s.check()
        if r == unknown:
            return dict(resultado="inconclusivo", k=k)
        if r == sat:
            m = s.model()
            l = next(l for l in range(k) if is_true(m.eval(ops[l],
model_completion=True)))
            ev = lambda e: is_true(m.eval(e, model_completion=True))
            ctes = [c for c in celulas if all(ev(tr[i][c] == tr[l][c]) for i in
range(l, k))]
            vivas = [c for c in ctes if ev(tr[l][c])]
            mortas = [c for c in ctes if not ev(tr[l][c])]
            return dict(resultado="contra-exemplo", k=k, l=l, vivas=vivas,
mortas=mortas,
                        modelo=m, traco=tr)
    return dict(resultado="inconclusivo", k=K)
```

3. Propriedade 1: "nunca se alcança o estado de erro"

Esta é uma propriedade de segurança, $\mathrm{G}\,\neg E$. Analisam-se dois casos:

- **(a)** com as células iniciais **sorteadas** (`SEED`), como no enunciado;
- **(b)** para **todas** as configurações das células iniciais (simbólicas). A propriedade só é *universalmente* verdadeira se nenhuma configuração leva ao erro.

Primeiro procura-se um contra-exemplo por BMC. Só no modo `sync` se tenta depois a prova por k-indução. No modo `async` o BMC encontra o erro em poucas transições, e uma prova por k-indução não faria sentido.

In [ ]:
```
def propriedade1(M, nome):
    t0 = time.time()
    r = bmc(M, M.erro, K_MAX)
    if r["resultado"] == "contra-exemplo":
        k = r["k"]
        print(f"[{nome}] G ¬erro é FALSA: o erro é alcançado em {k} transições
({time.time()-t0:.1f}s)\n")
```

```
        mostra_traco(M, r["modelo"], r["traco"][:k + 1])
        return "FALSA"
    if r["resultado"] == "inconclusivo":
        print(f"[{nome}] INCONCLUSIVO: o solver não respondeu a tempo no passo
{r['k']} ({time.time()-t0:.1f}s)")
        return "INCONCLUSIVO"
    if M.modo == "async":
        print(f"[{nome}] inconclusivo até K={K_MAX} (não provado)  ({time.time()-
t0:.1f}s)")
        return "SINCONCLUSIVO ATÉ K"
    res = kinducao(M, lambda s: Not(M.erro(s)), K_MAX)
    print(f"[{nome}] inconclusivo até K={K_MAX}; k-indução: {res}  ({time.time()-
t0:.1f}s)")
    return res[0]

print(f"===== modo {MODO} =====")
p1_conc = propriedade1(M_conc, "(a) células iniciais sorteadas")
```

```
===== modo sync =====
[(a) células iniciais sorteadas] INCONCLUSIVO: o solver não respondeu a tempo no pas
so 10 (12.4s)
```

In [ ]:
```
p1_simb = propriedade1(M_simb, "(b) todas as configurações de células iniciais")
```

```
[(b) todas as configurações de células iniciais] G ¬erro é FALSA: o erro é alcançado
em 3 transições (3.9s)

t=0                 t=1                 t=2                 t=3
..#.............    ..#.............    ..#.............    ..
#..............      
.XXX----------      .----------         .----------         .----------
#XXX----------      #---X------         #----------         #----------
.XXX----------      .--X----------      .--X----------      .----------
.----------         .-X-----------      .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.----------         .----------         .----------         .----------
.                   .                   .                   .
```

4. Propriedade 2: "nenhuma célula normal está permanentemente morta ou permanentemente viva"

A frase admite duas leituras formais:

1. **Leitura LTL (animação).** Em todos os traços, cada célula muda infinitas vezes:

$$\bigwedge_c \left(\mathrm{GF}\, s_c \wedge \mathrm{GF}\, \neg s_c\right).$$

   Um contra-exemplo é um laço em que alguma célula c é constante no ciclo, ou seja, um traço que satisfaz $\mathrm{FG}\, s_c$ ou $\mathrm{FG}\, \neg s_c$.
2. **Leitura "desde o início".** Nenhuma célula tem o mesmo valor em **todos** os estados alcançáveis:

$$\bigwedge_c \left(\mathrm{EF}\, s_c \wedge \mathrm{EF}\, \neg s_c\right).$$

   No modo `sync` verifica-se com um BMC incremental até $K_{MAX} = A + 1$: como esse BMC já visita todos os estados alcançáveis, uma célula que nunca muda está **provadamente** constante. No modo `async` não há um limite deste tipo; esta leitura é confirmada pela enumeração explícita (secção 6) para $N = 4$.

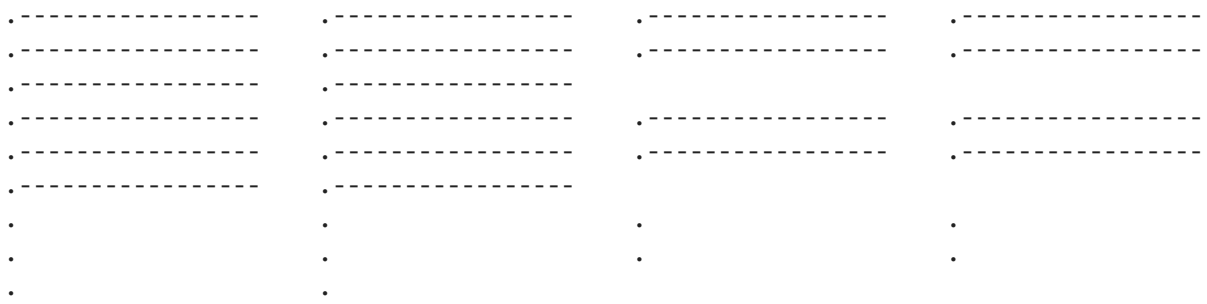
No modo `async` o não-determinismo levanta uma questão de **justiça** (*fairness*): um laço pode ignorar indefinidamente uma célula que podia mudar. Indica-se por isso se o ciclo encontrado é um **estado estável**. Um estado estável é um laço justo, onde nenhuma célula pode mudar.

In [ ]:
```
def propriedade2_ltl(M, nome):
    t0 = time.time()
    r = laco(M, K_MAX)
    if r["resultado"] == "inconclusivo":
        print(f"[{nome}] INCONCLUSIVO: o solver não respondeu a tempo em k=
{r['k']}  ({time.time()-t0:.1f}s)")
        return r
    if r["resultado"] == "inconclusivo":
        print(f"[{nome}] nenhum laço contra-exemplo até K={K_MAX}  ({time.time()-
t0:.1f}s)")
        return r
    k, l, m, tr = r["k"], r["l"], r["modelo"], r["traco"]
    estavel = (k - l == 1)
    print(f"[{nome}] FALSA: laço s{l}..s{k} (s{k} = s{l}), ciclo de comprimento
{k-l}"
          f"{' — estado ESTÁVEL (laço justo)' if estavel else ''}  ({time.time()-
t0:.1f}s)")
    print(f"   permanentemente vivas no ciclo : {len(r['vivas'])} células")
    print(f"   permanentemente mortas no ciclo: {len(r['mortas'])} células\n")
    mostra_traco(M, m, tr[:k + 1])
    return r

L2 = propriedade2_ltl(M_conc, "(a) células iniciais sorteadas")
```

```
[(a) células iniciais sorteadas] FALSA: laço s8..s9 (s9 = s8), ciclo de comprimento
1 — estado ESTÁVEL (laço justo)  (96.8s)
   permanentemente vivas no ciclo : 4 células
   permanentemente mortas no ciclo: 285 células

t=0                  t=1                  t=2                  t=3
t=4                  t=5                  t=6                  t=7
t=8                  t=9
#...##........#.#.#   #...##........#.#.#   #...##........#.#.#   #...##........#.#.
#   #...##........#.#.#   #...##........#.#.#   #...##........#.#.#   #...##........
#.#.#   #...##........#.#.#   #...##........#.#.#
.XXX--------------    .-----------------    .---XX------------    .---XX------------
.--X--------------    .--X--------------    .--X-X------------    .----X------------
.---XX------------    .---XX------------
.XXX--------------    .---X-------------    .-----------------    .---X-------------
.---XX------------    .---X-------------    .-----------------    .-----------------
.-----------------    .-----------------
.XXX--------------    .--X--------------    .--X--------------    .-----------------
.X----------------    .X----------------    .X----------------    .-----------------
.-----------------    .-----------------
#-----------------    #-X---------------    #X----------------    #XX---------------
#-X---------------    #-X---------------    #-----------------    #-----------------
#X----------------    #X----------------
#-----------------    #-----------------    #X----------------    #X----------------
#-X---------------    #-----------------    #X----------------    #X----------------
#X----------------    #X----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------
#-----------------    #-----------------    #-----------------    #-----------------
#-----------------    #-----------------    #-----------------    #-----------------
#-----------------    #-----------------
#-----------------    #-----------------    #-----------------    #-----------------
#-----------------    #-----------------    #-----------------    #-----------------
#-----------------    #-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------    .-----------------    .-----------------
.-----------------    .-----------------
#-----------------    #-----------------    #-----------------    #-----------------
#-----------------    #-----------------    #-----------------    #-----------------
#-----------------    #-----------------
#-----------------    #-----------------    #-----------------    #-----------------
#-----------------    #-----------------    #-----------------    #-----------------
#-----------------    #-----------------
```



In [ ]:

```
def propriedade2_inicio(M, K):
    """Modo sync: para cada célula, existe estado alcançável com o valor oposto
    ao inicial? BMC incremental partilhado por todas as células até K = A+1,
    que visita todos os estados alcançáveis. Devolve (sempre_viva, sempre_morta,
    inconclusivas)."""
    assert M.modo == "sync"
    pendentes = {c: bool(M.vivas0(*c)) for c in M.normais}
    inconcl = []
    s = novo_solver(); tr = [M.declare(0)]; s.add(M.init(tr[0]))
    for k in range(K + 1):
        if k > 0:
            tr.append(M.declare(k)); s.add(M.trans(tr[k - 1], tr[k]))
        for c in list(pendentes):
            s.push(); s.add(tr[k][c] == (not pendentes[c]))
            r = s.check()
            s.pop()
            if r == sat:
                del pendentes[c]
            elif r == unknown:
                inconcl.append(c); del pendentes[c]
        if not pendentes:
            break
    sempre_viva = [c for c, v0 in pendentes.items() if v0]
    sempre_morta = [c for c, v0 in pendentes.items() if not v0]
    return sempre_viva, sempre_morta, inconcl

if MODO == "sync":
    _t0 = time.time()
    _sv, _sm, _inc = propriedade2_inicio(M_conc, K_MAX)
    print(f"Leitura 'desde o início' (modo sync, células iniciais sorteadas, K=
{K_MAX})  ({time.time()-_t0:.1f}s)")
    print(f"   vivas em todos os estados alcançáveis : {len(_sv)} {_sv}")
    print(f"   mortas em todos os estados alcançáveis: {len(_sm)} células")
    if _inc: print("   inconclusivas (timeout):", _inc)
    print("   => propriedade", "FALSA" if (_sv or _sm) else ("VERDADEIRA" if not
_inc else "INCONCLUSIVA"))
else:
    print("Leitura 'desde o início': não é corrida no modo async (sem limite útil
para K).")
    print("Ver a enumeração explícita (secção 6) com N=4: a propriedade é
VERDADEIRA.")
```

```
Leitura 'desde o início' (modo sync, células iniciais sorteadas, K=10)  (14.0s)
   vivas em todos os estados alcançáveis : 0 []
   mortas em todos os estados alcançáveis: 272 células
   => propriedade FALSA
```

In [ ]:
```
L2s = propriedade2_ltl(M_simb, "(b) células iniciais simbólicas")
```

```
[(b) células iniciais simbólicas] FALSA: laço s2..s3 (s3 = s2), ciclo de comprimento
1 — estado ESTÁVEL (laço justo)  (9.1s)
   permanentemente vivas no ciclo : 10 células
   permanentemente mortas no ciclo: 279 células

t=0                  t=1                  t=2                  t=3
#....##.##.#..####.  #....##.##.#..####.  #....##.##.#..####.  #....##.##.#..###
#.
.XXX--------------   .--XX----------XX-   .--XX---------X--X   .--XX---------X--X
.XXX--------------   .---X-------------   .-X-X-------------   .-X-X-------------
.XXX--------------   .X-X--------------   .X-X--------------   .X-X--------------
.-----------------   .XX---------------   .XX---------------   .XX---------------
#-----------------   #-----------------   #-----------------   #-----------------
#-----------------   #-----------------   #-----------------   #-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.-----------------   .-----------------   .-----------------   .-----------------
.                    .                    .                    .
```

5. Os dois modos

Cada modo corre **separadamente**: muda-se `MODO` na célula de parâmetros e o Marimo volta a correr as células que dependem dele. Assim cada modo usa a profundidade que lhe convém:

modo	K_MAX	propriedades verificadas
sync	$A + 1$ (calculado)	P1 (a) e (b), P2 LTL, P2 "desde o início"
async	K_ASYNC	P1 (a) e (b) por BMC, P2 LTL

6. Validação independente por enumeração explícita

Este verificador não usa o Z3. Os estados são *bitmasks* das N^2 células normais e calculam-se por BFS todos os estados alcançáveis. O resultado tem de coincidir com o do Z3.

Para $N = 4$ e o modo `sync` percorrem-se também **todas** as $2^{2N+3} = 2048$ configurações das células iniciais.

In [ ]:
```
for _modo in ("sync", "async"):
    if _modo == "async" and N > 4:
        print("async: enumeração explícita omitida para N > 4 (2^(N^2) estados)");
continue
    _r = explicito(N, INICIAIS, _modo)
    print(f"{_modo:5}: {_r['alcancaveis']} estados alcançáveis (profundidade
{_r['profundidade']}); erro em "
          f"{'nunca' if _r['erro'] is None else str(_r['erro']) + ' transições'};
"
          f"sempre vivas {len(_r['sempre_viva'])}; sempre mortas
{len(_r['sempre_morta'])}")

if N == 4:
    _ini = [(i, j) for i in range(N + 2) for j in range(N + 2) if i == 0 or j ==
0]
    _cnt, _dmin = 0, None
    for _bits in itertools.product((0, 1), repeat=len(_ini)):
        _e = explicito(N, dict(zip(_ini, _bits)), "sync")["erro"]
        if _e is not None:
            _cnt += 1; _dmin = _e if _dmin is None else min(_dmin, _e)
    print(f"\nsync, todas as {2**len(_ini)} configurações: o erro é alcançável em
{_cnt} "
          f"({100*_cnt/2**len(_ini):.1f}%), no mínimo em {_dmin} transições")
```

```
sync : 9 estados alcançáveis (profundidade 8); erro em nunca; sempre vivas 0; sempre
mortas 272
async: enumeração explícita omitida para N > 4 (2^(N^2) estados)
```

7. Conclusões

Resultados com $N = 17$, `SEED = 2026`, cada modo corrido separadamente:

	`sync` ($K_{MAX} = A + 1 = 10$)	`async` ($K_{MAX} =$ `K_ASYNC` $= 12$)
P1 (sorteadas)	**verdadeira**: provada por k-indução (diâmetro de recorrência, $k = 10$). A trajetória tem só 9 estados e cai num ponto fixo.	**falsa**: o erro é alcançado em 9 transições, matando as células uma a uma
P1 (todas as configurações)	**falsa**: há configurações que levam ao erro em 3 transições	**falsa**: erro em 9 transições
P2, leitura LTL	**falsa**: ponto fixo em $t = 8$	**falsa**: estado estável (laço justo) em 5 transições
P2, leitura "desde o início"	**falsa**: 272 das 289 células normais estão mortas em todos os estados alcançáveis	não corrida (sem limite útil para K); verdadeira para $N = 4$ pela enumeração explícita

Validação com $N = 4$, onde a enumeração explícita é possível nos dois modos: o Z3 dá os mesmos resultados que a BFS. No modo `sync` há 12 estados alcançáveis, P1 provada, ponto fixo em $t = 11$ e as células $(4, 1)$ e $(4, 4)$ sempre mortas. 10,1% das 2048 configurações da moldura levam ao erro, no mínimo em 3 transições. No modo `async` o erro surge em 9 transições, há um estado estável em 5, e a leitura "desde o início" da P2 é **verdadeira** nos 65 144 estados alcançáveis.

- **P1 no modo `async`.** Como cada transição escolhe *qual* célula muda, o sistema pode sempre escolher matar uma célula com menos de 2 vizinhos vivos. Assim é muito mais fácil chegar ao erro do que no modo síncrono, e o número de transições (9 = as 9 células vivas iniciais) não depende de N.
- **P1 depende das células iniciais.** Com o sorteio `SEED=2026` o erro nunca é alcançado no modo `sync`, mas a versão simbólica mostra que isto não é verdade para todos os sorteios.
- **P2.** Num sistema finito e determinista (`sync`), todo o traço acaba num ciclo, e com estas fronteiras o ciclo é um ponto fixo. Por isso a leitura LTL é falsa. A maioria das células, longe do quadrado inicial, nunca chega a nascer, o que torna falsa também a leitura "desde o início". No modo `async` *cada traço individual* acaba igualmente num estado estável.
- **Escolha de `K_MAX`.** No modo `sync` não foi adivinhado: a enumeração explícita dá A estados alcançáveis e $K_{MAX} = A + 1$ garante que o diâmetro de recorrência fecha a prova. No modo `async` basta `K_ASYNC = 12` para os contra-exemplos (9 e 5 transições); não se tentam provas.
- **Codificação.** A regra com `PbEq` e a transição `async` linear tornam $N = 17$ tratável. Com a contagem `Sum(If(...))` e a transição quadrática, mesmo $K_{MAX} = 5$ demorava mais de um minuto.
- **Timeouts.** Uma resposta `unknown` do Z3 é sempre reportada como **inconclusiva**. Nenhum dos resultados acima depende de um timeout.
- **Contagem das células.** O enunciado indica $2N + 1$ iniciais e $2N + 3$ finais. Seguimos a convenção do notebook do professor ($2N + 3$ iniciais e $2N + 1$ finais), contando $(0, N + 1)$ e $(N + 1, 0)$ como iniciais.

8. Impacto de pequenas modificações nos requisitos

O código foi escrito para que cada alteração plausível do enunciado seja **uma linha** na célula de parâmetros ou em `GameOfLife`:

modificação	onde	efeito esperado
outra regra (ex.: *HighLife* B36/S23, *Seeds* B2/S)	`NASCE`, `SOBREVIVE`	o modelo não muda. Com B2/S, nenhuma célula sobrevive e o erro torna-se muito mais provável

modificação	onde	efeito esperado
outra configuração inicial	`VIVAS_T0`	só muda `init`
outra probabilidade p	`P_VIVA`	só afeta o caso (a). O caso (b), simbólico, já cobre todas as configurações
células finais vivas, ou aleatórias como as iniciais	`self.cte` para `finais`	aumenta o número de vizinhos vivos junto à fronteira. A análise é a mesma
grelha toroidal (sem células iniciais/finais)	`self.viz` com índices `mod N`	o sistema fica fechado. Desaparecem as constantes e a simetria aumenta
k células a mudar por transição (em vez de 1 ou todas)	`trans` : `AtMost(m, k)` / `AtLeast(m, k)`	interpola entre `async` e `sync`
justiça forte no modo `async`	`laco` : exigir que cada célula *habilitada* mude no ciclo	elimina contra-exemplos artificiais à P2. Os estados estáveis continuam a ser contra-exemplos
N maior	`N`	o modo `sync` escala bem: tem poucos estados alcançáveis e a codificação booleana resolve-se por propagação. No modo `async` o espaço de estados é 2^{N^2}; o BMC continua a encontrar contra-exemplos curtos, mas provas exigiriam um invariante reforçado ou IC3/PDR

9. Testes

Testes automáticos ao modelo e aos métodos de verificação. Usam grelhas pequenas ($N = 4$ e $N = 5$), por isso correm em poucos segundos. Podem ser corridos de duas formas:

- dentro do notebook: a última célula corre todos os testes e mostra o resultado;
- no terminal: `pytest game_of_life.py` (o pytest encontra sozinho as funções `test_…`).

teste	o que verifica
`test_contagem_celulas`	N^2 normais, $2N + 3$ iniciais e $2N + 1$ finais, sem sobreposições
`test_vizinhos`	toda a célula normal tem exatamente 8 vizinhos
`test_regra_igual_contagem`	a regra com `PbEq` dá o mesmo que contar os vizinhos em Python, em estados aleatórios
`test_regra_bloco` / `test_regra_blinker`	padrões clássicos: o bloco é estável e o *blinker* oscila

teste	o que verifica
`test_async_muda_uma_celula`	no modo `async` cada sucessor difere numa única célula, uma das que a regra deixa mudar
`test_async_stuttering`	num estado estável o único sucessor `async` é o próprio estado
`test_profundidade_sync`	no modo `sync` a profundidade é $A-1$ e o BMC até $A+1$ não acha o erro
`test_z3_igual_explicito`	o BMC e a enumeração explícita dão a mesma distância ao erro, em vários sorteios
`test_resultados_enunciado`	os resultados das conclusões com $N=4$, `SEED=2026`

In [ ]:
```python
# ---------- funções auxiliares dos testes ----------
def iniciais_mortas(N):
    """Células iniciais todas mortas: isola o padrão da fronteira."""
    R = range(N + 2)
    return {(i, j): False for i in R for j in R if i == 0 or j == 0}


def modelo_com(N, modo, vivas, iniciais=None):
    """GameOfLife cujo estado inicial tem vivas exatamente as células 'vivas'."""
    return GameOfLife(N, modo, iniciais if iniciais is not None else
iniciais_mortas(N),
                      vivas0=lambda i, j: (i, j) in vivas)


def sucessores(M):
    """Todos os sucessores do estado inicial (conjuntos de células vivas)."""
    s, p = M.declare(0), M.declare(1)
    sol = Solver(); sol.add(M.init(s), M.trans(s, p))
    res = []
    while sol.check() == sat:
        m = sol.model()
        res.append({c for c in M.normais if is_true(m.eval(p[c],
model_completion=True))})
        sol.add(Or([p[c] != m.eval(p[c], model_completion=True) for c in
M.normais]))
    return res


def regra_python(M, iniciais, vivas, c):
    """B3/S23 calculada diretamente em Python (sem Z3)."""
    n = sum(1 for d in M.viz[c] if (d in vivas) or iniciais.get(d, False))
    return n in M.nasce or (c in vivas and n in M.sobrevive)


def avalia(M, s, vivas, formula):
    """Valor de uma fórmula num estado concreto."""
    sol = Solver()
    sol.add([s[c] == BoolVal(c in vivas) for c in M.normais])
```

```
    assert sol.check() == sat
    return is_true(sol.model().eval(formula, model_completion=True))
```

In [ ]:

```
def test_contagem_celulas():
    for n in range(4, 8):
        M = GameOfLife(n, "sync")
        normais, iniciais, finais = set(M.normais), set(M.iniciais), set(M.finais)
        assert len(normais) == n * n
        assert len(iniciais) == 2 * n + 3
        assert len(finais) == 2 * n + 1
        assert not (normais & iniciais) and not (normais & finais) and not
(iniciais & finais)
        assert len(normais | iniciais | finais) == (n + 2) ** 2


def test_vizinhos():
    M = GameOfLife(5, "sync")
    todas = set(M.normais) | set(M.iniciais) | set(M.finais)
    for c in M.normais:
        assert len(M.viz[c]) == 8 and len(set(M.viz[c])) == 8
        assert c not in M.viz[c]
        assert all(v in todas for v in M.viz[c])


def test_regra_igual_contagem():
    rng = random.Random(1)
    for seed in (3, 5):
        ini = sorteia_iniciais(5, 0.5, seed)
        M = GameOfLife(5, "sync", ini)
        s = M.declare(0)
        for _ in range(5):
            vivas = {c for c in M.normais if rng.random() < 0.4}
            for c in M.normais:
                assert avalia(M, s, vivas, M.regra(s, c)) == regra_python(M, ini,
vivas, c)


def test_regra_bloco():
    bloco = {(2, 2), (2, 3), (3, 2), (3, 3)}
    for modo in ("sync", "async"):
        assert sucessores(modelo_com(5, modo, bloco)) == [bloco]


def test_regra_blinker():
    horizontal = {(3, 2), (3, 3), (3, 4)}
    vertical = {(2, 3), (3, 3), (4, 3)}
    assert sucessores(modelo_com(5, "sync", horizontal)) == [vertical]
    assert sucessores(modelo_com(5, "sync", vertical)) == [horizontal]


def test_async_muda_uma_celula():
    vivas = {(1, 1), (1, 2), (2, 1), (3, 3), (4, 2)}
    M = modelo_com(4, "async", vivas)
    podem_mudar = {c for c in M.normais if regra_python(M, {}, vivas, c) != (c in
vivas)}
    sucs = sucessores(M)
```

```
    assert len(sucs) == len(podem_mudar) > 0
    for t in sucs:
        dif = t ^ vivas
        assert len(dif) == 1 and dif <= podem_mudar


def test_async_stuttering():
    bloco = {(1, 1), (1, 2), (2, 1), (2, 2)}
    assert sucessores(modelo_com(4, "async", bloco)) == [bloco]


def test_profundidade_sync():
    ini = sorteia_iniciais(4, 0.5, 2026)
    r = explicito(4, ini, "sync")
    assert r["profundidade"] == r["alcancaveis"] - 1      # trajetória única, sem
ramificações
    M = GameOfLife(4, "sync", ini)
    assert bmc(M, M.erro, r["alcancaveis"] + 1)["resultado"] == "inconclusivo"


def test_z3_igual_explicito():
    for seed in (1, 7, 42, 2026):
        ini = sorteia_iniciais(4, 0.5, seed)
        M = GameOfLife(4, "sync", ini)
        e = explicito(4, ini, "sync")
        r = bmc(M, M.erro, e["alcancaveis"] + 1)
        assert (r["k"] if r["resultado"] == "contra-exemplo" else None) ==
e["erro"]


def test_resultados_enunciado():
    ini = sorteia_iniciais(4, 0.5, 2026)
    Ms = GameOfLife(4, "sync", ini)
    K = explicito(4, ini, "sync")["alcancaveis"] + 1
    assert kinducao(Ms, lambda s: Not(Ms.erro(s)), K)[0].startswith("PROVADO")
    Msimb = GameOfLife(4, "sync")
    assert bmc(Msimb, Msimb.erro, K)["k"] == 3
    Ma = GameOfLife(4, "async", ini)
    assert bmc(Ma, Ma.erro, 12)["k"] == 9
    assert explicito(4, ini, "async")["erro"] == 9
```

```
.........                                                                [100%]
================================== Overview ====================================
Passed Tests:
✓ game_of_life.py::test_contagem_celulas
✓ game_of_life.py::test_vizinhos
✓ game_of_life.py::test_regra_igual_contagem
✓ game_of_life.py::test_regra_bloco
✓ game_of_life.py::test_regra_blinker
✓ game_of_life.py::test_async_muda_uma_celula
✓ game_of_life.py::test_async_stuttering
✓ game_of_life.py::test_profundidade_sync
✓ game_of_life.py::test_z3_igual_explicito
✓ game_of_life.py::test_resultados_enunciado

Summary:
Total: 10, Passed: 10, Failed: 0, Errors: 0, Skipped: 0
```

In [ ]:
```python
# ---------- correr os testes dentro do notebook ----------
_testes = [test_contagem_celulas, test_vizinhos, test_regra_igual_contagem,
test_regra_bloco,
           test_regra_blinker, test_async_muda_uma_celula, test_async_stuttering,
           test_profundidade_sync, test_z3_igual_explicito,
test_resultados_enunciado]
_falhas = 0
for _t in _testes:
    try:
        _t()
        print(f"  OK     {_t.__name__}")
    except AssertionError as _e:
        _falhas += 1
        print(f"  FALHOU {_t.__name__} {_e}")
print(f"\n{len(_testes) - _falhas}/{len(_testes)} testes passaram")
```

```
 OK     test_contagem_celulas
 OK     test_vizinhos
 OK     test_regra_igual_contagem
 OK     test_regra_bloco
 OK     test_regra_blinker
 OK     test_async_muda_uma_celula
 OK     test_async_stuttering
 OK     test_profundidade_sync
 OK     test_z3_igual_explicito
 OK     test_resultados_enunciado

10/10 testes passaram
```

Utilização de LLMs

Neste trabalho usámos o Claude, através de claude.ai.

O LLM ajudou-nos a estruturar o início do projeto e a perceber a estrutura pedida no enunciado.

Nenhuma resposta do LLM foi aceite sem ser executada e testada. Link: https://claude.ai/share/9ce9eff8-7cad-495e-a01c-86717367da33